COMPILE ONCE, QUERY MANY: REUSABLE EXECUTABLE MODELS FOR PROCESS-RECORD QUESTION ANSWERING

Anonymous Authors[1]

ABSTRACT

Process-record documents (e.g., logs, tickets, runbooks, and lab notes) describe actions taken and observed outcomes, and teams ask many follow-up questions about them. Retrieval-style document QA can answer factual queries, but counterfactual "what-if" questions require inferring the process that generated the record; rebuilding that process for each query wastes inference, even for code-writing agents. We introduce OTF-Tools, a system that compiles a process-record document into reusable executable code: it preserves complete tables with source locations, synthesizes programs to extract inputs and simulate outcomes, and verifies each program by reproducing recorded outcomes before reuse. Later questions can change the inputs and execute the stored code, while a reading fallback handles questions without a suitable simulator. Across 3,066 distinct questions in physics, manufacturing, IT operations, and a finance fallback control, using three open-weight LLMs and seven baselines, OTF-Tools lies on the measured accuracy–model-token Pareto frontier in all 14 comparable settings. On physics with Qwen3.6-27B, it achieves 74.9% accuracy at 8.1k amortized tokens per question, compared with PageIndex's 76.7% at 33.7k tokens. Executable replay raises counterfactual accuracy by 4.9 percentage points over reading the relevant records alone. Accuracy changes by less than one point when the same questions are placed in a document fifteen times longer; one accepted physics program serves fifteen documents and recovers its setup cost against full-context prompting after three questions. These results demonstrate that keeping process records accessible and reusing executable process code can substantially reduce repeated LLM inference, while the experiments also identify when program synthesis or execution is less effective.

1 INTRODUCTION

Many operational documents record a sequence of events rather than independent facts. A manufacturing ledger, for example, lists orders, production, and shipments; accompanying policy text may explain how those orders are processed. A question about what actually shipped can be answered by reading the ledger. A question about what *would have shipped if an order had never arrived* requires computing how the process would behave under different inputs. We study such questions as *process-record question answering* (PRQA).

Consider a plant where products X and Y have separate inventories but share limited dispatch capacity. An order for Y arrives on day 38; an order for X arrives on day 40 and ships only partially. To determine whether X would have shipped in full without Y, the system must find the orders and relevant production events, remove Y, and apply the dispatch rules to the resulting sequence. The answer is not a sentence waiting to be retrieved: it is an outcome that must be recomputed. Figure 2 follows this example through OTF-Tools.

A language model can read the relevant pages and reconstruct the process for each question. A tool-using agent can even write a short program, but doing so again for the next question repeats work. Retrieval-augmented generation (RAG) reduces reading by selecting relevant text chunks. The chunk-based RAG baseline in our evaluation, however, does not provide the complete tables and source-addressed records that our program uses. OTF-Tools instead accesses the record group identified by the question and preserves its original structure. Table-aware retrieval could provide similar access; the more distinctive contribution is *building process code once and using it for many questions*.

We introduce OTF-Tools, which prepares a document for repeated questions in three steps. **First, it organizes the records.** A Navigation Map preserves the document's sections and tables, including their rows, columns, and source

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

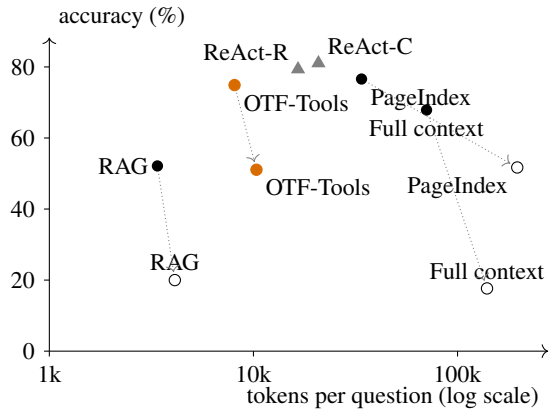


Figure 1. Accuracy against tokens per question (setup cost included) for physics questions with Qwen3.6-27B. Filled circles show all E1 physics questions on documents of about 54k tokens; open circles show the E2 explanatory and counterfactual subset repackaged into one document fifteen times longer. Dotted arrows connect the respective settings, not matched question sets; the paired E1/E2 comparison is given in Table 6. Triangles show query-time agents on the five-document E1-core subset.

locations. Each benchmark question already names the section it concerns, so OTF-Tools can retrieve the entire relevant record group directly rather than search unrelated sections. **Second, it builds and checks a program.** An LLM writes functions to read the inputs and observed outcomes, together with a simulator that computes outcomes from those inputs. OTF-Tools then hides selected recorded outcomes during a test run and checks whether the generated program can reproduce them. **Third, it reuses the program.** For a new question, the system changes the relevant inputs, runs the same program, and uses the computed outcomes to formulate an answer. An accepted program can also be tested on a new document with the same record structure. When execution is unavailable, the system answers from the relevant section instead.

This division of work offers two complementary benefits. Direct access to a known section limits the evidence sent to the LLM; executing stored code avoids reconstructing process rules for each question. The difference matters because many questions need only the first benefit, whereas predictive and counterfactual questions can also benefit from the second.

Our evaluation spans 3,066 distinct questions over physics, manufacturing, IT operations, and financial reports used to test the reading fallback, with three open-weight models and seven baselines. We contribute:

1. **A reusable execution architecture** that extracts process records, generates and tests executable models, and reuses accepted programs across questions and structurally compatible documents (§3).
2. **A component-level explanation of accuracy gains.** With Qwen3.6-27B on physics, scoped reading achieves 73.7% overall accuracy. Executable replay raises it to 74.9% and improves counterfactual accuracy by 4.9 percentage points (§5.2).
3. **A systems evaluation of accuracy and amortized inference.** On physics, OTF-Tools uses 8.1k model tokens per question versus 33.7k for PageIndex at a 1.8-point accuracy difference. We measure reuse, break-even cost, document aggregation, latency, and failure cases (§5).

The results show how structured evidence access and reusable computation jointly reduce the cost of answering repeated questions about process records.

2 Process-Record Question Answering

2.1 Documents, records, and questions

A *process record* describes what happened as a system operated: an order arrived, a task started, a collision occurred, or a shipment completed. A *process-record document* packages such observations in tables or text. It may also contain *process documentation* describing operating rules. For example, a dispatch policy specifies how shipping capacity is allocated, whereas a ledger records which orders actually shipped.

We call the set of records for one process execution a *scope*—such as one plant over fourteen days or one physics video. Each benchmark question provides the scope it concerns. Let D denote a document containing scopes $\mathcal{S}(D)$. Within scope s, let X_s be the process inputs (incoming orders), θ_s its parameters (dispatch capacity), and Y_s its observed outcomes (shipments). Their relationship can be expressed as

$$Y_s = f_s(X_s, \theta_s), \tag{1}$$

where f_s represents the process. OTF-Tools attempts to construct executable code that reproduces relevant behavior of f_s. In manufacturing and IT, the benchmarks provide written policies and simulated outcomes; physics instead provides observed motion and event records.

2.2 Computing an answer

We evaluate four types of questions (Yi et al., 2020). A *descriptive* question asks for a recorded fact. A *predictive* question asks what happens next. An *explanatory* question asks which condition an outcome depends on. A *counterfactual* question changes an input and asks how the outcome changes. For a counterfactual, δ_q denotes the change—for instance, removing order Y. A generated program P com-

putes

$$\widehat{Y}_{s,q} = \mathrm{Sim}_P(X_s \oplus \delta_q, \theta_s; t_0, t_1), \tag{2}$$

where $\oplus$ changes the input records and $[t_0, t_1]$ is the simulated time interval. The result $\widehat{Y}_{s,q}$ is a newly computed outcome, not a copy of the original shipment record.

2.3 Amortizing the computation

A single dispatch policy or scheduler may govern hundreds of questions and several documents. Reconstructing its rules for each question repeats model work. OTF-Tools instead incurs a one-time program-construction cost B, followed by the cost q_i of answering each new question:

$$C(Q) = B + \sum_{i=1}^{Q} q_i. \tag{3}$$

We assess whether the resulting accuracy justifies the total model-token cost, including unsuccessful program builds. PRQA shares process evidence with process mining, but its objective is answering questions rather than discovering or checking process models (van der Aalst, 2016).

3 DESIGN

OTF-Tools separates work performed *before* questions arrive from work performed *for each* question (Figure 2). The first part reads and organizes the document, writes process code, and tests it. The second part finds the question's records, changes the relevant inputs, and executes accepted code. We use the manufacturing example to explain these operations.

3.1 Keeping complete records together

First, a deterministic parser constructs a *Navigation Map* from the PDF. Each section retains its title, page range, and contents. Tables retain their rows and columns, and each cell or line has an address pointing to its original location. The map therefore keeps the order table and any accompanying dispatch policy accessible in their original form. Building it requires no LLM calls.

Each question in our evaluation already specifies the relevant section, called its *scope*. OTF-Tools retrieves that section directly. The evaluated RAG baseline instead ranks fixed-length chunks of text for the question: it may retrieve the named order but omit production events or other orders that affect the result. By preserving the full record group and its table structure, the Navigation Map supplies related inputs and outcomes together. A table-aware, metadata-filtered retriever could provide similar evidence access; the main reason for our representation is to support consistent record extraction and later computation.

3.2 Writing code to read and replay the process

The LLM generates two components. The first consists of *reader functions*, which turn document rows into records with a time, event type, entities, values, and source address. In the running example, a reader extracts the day-38 order as an input, identifies shipping capacity as a parameter, and extracts actual shipments as outcomes. These are different kinds of evidence: the inputs and policy describe what the process can do, while the outcomes record what it did.

The second component is a *simulator*, a program that takes these inputs and computes the resulting outcomes. Every generated simulator has the same interface:

```
simulate(doc, scope, inputs, params,
start, horizon)
```

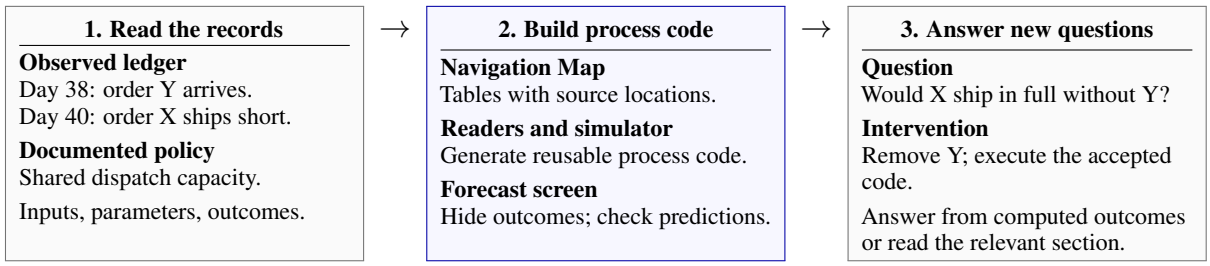
Here `inputs` contains the events to simulate, `params` contains process parameters, and `start` and `horizon` specify the time interval. The program returns computed outcome records. It must use the supplied `inputs` rather than reread the original orders from the PDF. Otherwise removing order Y from the inputs would have no effect on the execution.

To generate these functions, the LLM receives the document's front matter, a description of its table layout, and example records. It can test three to eight short snippets in a restricted interpreter. The prompts describe the record format and simulator interface without naming the application domain.

3.3 Checking whether the code reproduces recorded behavior

Generated code is useful only if it computes outcomes rather than copies them. OTF-Tools therefore performs a *forecast-consistency check* before storing a program. It makes a testing copy of the records in which outcomes after an early cutoff are hidden, while later input events remain available. It runs the simulator and compares the predicted events with the hidden ones, matching event types and entities with time and value tolerances detailed in the supplement. The mean record-level F1 over up to six scopes must reach 0.4.

Three additional tests detect shortcuts. The input reader must produce the same inputs with and without outcome masking; the outcome reader must recover almost none of the hidden outcomes (F1 at most 0.05); and changing an input must change at least one predicted outcome. A program that fails receives a report and may be revised within a fixed number of attempts. These tests assess agreement with recorded behavior and sensitivity to input changes. They do not independently establish correctness for all counterfactual interventions (§7).



Core distinction: a process-record document stores observed evidence, while a process program computes alternative outcomes.

Figure 2. How OTF-Tools prepares a process record and reuses the resulting code. The day-38/day-40 example illustrates the stages: retain the order ledger, write and test a simulator, then remove an order and compute new shipments. When execution is unavailable, the system answers by reading the relevant records. Unsuccessful build attempts are included in setup cost.

3.4 Using one program on other documents

A new plant ledger may contain different orders but use the same table columns and dispatch policy. To avoid regenerating code, OTF-Tools stores accepted programs under a *structural signature* derived from section titles, column names, and record types. When another document has a matching signature, the system tests the stored program on that document's records. If it passes, the same program is reused; otherwise a new program is generated. The signature finds candidates, while the new-document check determines whether they can be used. All checks and unsuccessful attempts count toward setup cost.

3.5 Answering a question by changing the inputs

At question time, the first LLM call interprets the question and, when needed, specifies the input change. For the example, the change is simply *remove order Y from day 38*. The engine applies this change to the extracted records and calls the accepted simulator. It can also add an event, modify a value or time, or change a parameter. The simulator receives the modified records, not the question text.

To reduce repeated computation, the engine keeps the original history until the first recorded outcome involving the changed entity and simulates forward from there. This choice of restart point is a heuristic (§7). The final LLM call compares the original and simulated outcomes and writes the answer. Predictive questions simulate documented future inputs; explanatory questions test candidate removals. For descriptive questions or when no suitable simulator is available, OTF-Tools answers directly from the retrieved section. In the evaluated configurations, each question uses two or three LLM calls.

4 Implementation

OTF-Tools is implemented in approximately 1.8k lines of Python. It calls each language model through an OpenAI-compatible interface, so the same client records token use during program construction, question interpretation, and answer generation. We include all of these model calls in the cost measurements, rather than counting only the final answers.

Limits on program generation. The LLM has up to three attempts to produce the reader functions. For a simulator, it can submit at most five candidates in each of up to three conversations, receiving test feedback after a failed candidate. These limits cap the expense of program construction. If no program passes, the system still answers questions by reading the relevant section.

Running generated code. Each generated program runs in a separate interpreter process with a 20-second timeout. Only selected standard-library modules and NumPy are permitted; static checks reject filesystem, process, and interpreter operations. This reduces accidental unsafe access but does not provide a security guarantee against hostile generated code.

Reusing computed results. When two questions request the same execution, the system need not rerun the simulator. Its cache key includes the document, program, scope, full input list, parameters, start time, and horizon. An intervention that removes an order therefore has a different key from an unchanged execution, preventing the cache from returning the original outcome for the modified question.

5 Evaluation

We evaluate three questions: Does running a compiled program improve answers beyond reading the right records? How much does compilation cost relative to answering questions with an LLM each time? And how do those costs change when documents grow or a program is reused? We compare seven baselines, examine individual system components, and report accuracy together with model-token cost.

5.1 Setup

Workloads. Table 1 summarizes three process-record workloads and one reading-only control. *Physics* turns CLEVRER video annotations into documents containing object descriptions, motion measurements, and events (Yi et al., 2020). Each video is one scope; the document includes its first 128 observed frames. The main experiment, *E1*, groups 300 videos into 15 documents of approximately 54k tokens, with 2,166 questions. To test the effect of document length, *E2* combines the same video records into one 784-page document of about 838k tokens and repeats the 1,067 explanatory and counterfactual questions. Thus E2 changes how records are packaged, not which questions are asked.

Manufacturing uses 221 days of real production and order inputs from SupplyGraph (Wasi et al., 2024). A written dispatch policy determines simulated inventory, shipments, and backorders, with orders served oldest-first under a shared daily shipping capacity. *IT operations* uses real task and failure inputs from the Google 2011 cluster trace (Reiss et al., 2011); a documented priority-based scheduler computes outcomes for eight simulated machines. Both workloads require answers about altered input histories. *Finance* uses annual-report tables and text paired with TAT-HQA hypothetical questions and TAT-QA factual questions (Li et al., 2022; Zhu et al., 2021). These reports do not provide the time-indexed process expected by the implemented simulator and therefore test the reading fallback.

Manufacturing and IT labels are generated by reference simulators under the stated policies. The project reports an independently written cross-check of the printed records and labels: A second, separately implemented simulator, written from the stated policy and fed only values parsed back from the PDF, reproduces all 5,488 manufacturing ledger rows and all 9,104 IT task-log rows, and re-derives all 600 answer labels. Physics and finance retain their source labels. Simply copying factual outcomes answers none of the manufacturing or IT counterfactual questions correctly.

Models. We evaluate gpt-oss-20b (medium reasoning) (OpenAI, 2025), Qwen3.6-27B (Qwen Team, 2026), and Gemma-4-31B (Gemma Team, 2026), with thinking enabled for the latter two. Each model is pinned to one hosted provider. All systems use the same model within a comparison, including for their indexing or generation calls. The experimental context limit is 131,072 tokens and the maximum output is 16,384 tokens per call. The supplement records provider and sampling details. One ReAct-C physics run uses a local Qwen checkpoint and is identified in the results.

Comparison systems. *Full context* sends the document in a single prompt; on E2, only the first 106 pages fit. The evaluated *RAG* baseline retrieves five 512-token text chunks labeled with their section paths; the baseline embeds 512-token chunks with a 64-token overlap using the o200k tokenizer, cuts them within outline sections and prefixes them with their section path via the fastembed 0.9.0 ONNX build of bge-small-en-v1.5, ranks all chunks by exact cosine similarity, and passes the top five in rank order (Xiao et al., 2024). *PageIndex* (v0.4.0) searches a tree of document sections (Zhang & Tang, 2025), while *GraphRAG* uses a graph of entities and their summaries (Edge et al., 2024). Three agents can make up to twelve calls per question: *ReAct-R* searches and reads sections (Yao et al., 2023); *ReAct-C* can additionally execute Python code using OTF-Tools' Navigation Map and interpreter; and *ReCode* recursively builds code from plans (Yu et al., 2025). ReAct-C is especially important because it tests the cost of writing code for each question instead of building reusable code once. The physics agent and GraphRAG runs use a fixed five-document subset (*E1-core*, 694 questions), on which OTF-Tools is evaluated separately for a matched comparison. A hand-written physics simulator provides an additional oracle control.

Evaluation protocol. Prompts and settings were chosen on 20 separate physics development videos (308 questions), with no more than three configurations per system; no additional tuning was performed on the other domains. Gemma's GraphRAG and agent settings were inherited from Qwen. Some Gemma GraphRAG and ReCode runs are partial, and Gemma E2 was not completed; the relevant comparisons are marked or omitted. Every system receives the same scope identifier, question, answer choices, format instructions, and scorer, but never a question-type or answer label.

Accuracy and efficiency. Descriptive answers are scored by exact match; CLEVRER multiple-choice answers must select every correct option; and finance uses TAT-QA exact match with scale. Invalid or missing answers count as incorrect. Differences use paired 95% cluster-bootstrap intervals (2,000 resamples of scopes) (Efron & Tibshirani, 1993). Each configuration ran once without fixed seeds, so these intervals describe variation across questions, not independent program-generation runs.

Model-token cost includes both input and output, including reasoning tokens. Setup cost B includes index creation, successful and failed program attempts, and program rechecks; per-question cost is q_i (Equation 3). We distribute setup over the workload's questions and compute the point at which OTF-Tools' total token cost becomes lower than a baseline:

$$Q^* = \frac{B_{\mathrm{OTF}} - B_{\mathrm{base}}}{\overline{q}_{\mathrm{base}} - \overline{q}_{\mathrm{OTF}}}. \tag{4}$$

This expression applies when the baseline costs more per question; if OTF-Tools also has lower setup cost, it is already cheaper. We additionally report query latency on

Table 1. Evaluation workloads. D/E/P/C denote descriptive, explanatory, predictive, and counterfactual questions. E1 and the other three domains contain 3,066 distinct questions in total; E2 repeats 1,067 physics questions. Manufacturing and IT inputs are real; their outcomes are computed under the written policy.

Domain	Records and reference answers	Documents	D / E / P / C
Physics	Synthetic CLEVRER trajectories rendered as text; CLEVRER labels.	E1: $15 \times \sim 53$ pp.	882 / 513 / 217 / 554
	Same trajectories and questions repackaged for scale.	E2: 1×784 pp.	– / 513 / – / 554
Manufacturing	Real production and orders; simulated dispatch, inventory, and backorders.	5×42–62 pp.	75 / 75 / 75 / 75
IT operations	Real cluster inputs; simulated scheduling and execution on eight machines.	5×44–54 pp.	75 / 75 / 75 / 75
Finance	Annual-report tables and text; TAT-HQA and TAT-QA labels.	3×95–105 pp.	150 / – / – / 150

Table 2. Does executing a program improve accuracy beyond reading the identified section? Both systems use the same Navigation Map and evidence (Qwen3.6-27B, 2,166 physics E1 questions). C denotes counterfactual accuracy; model-token costs include setup. The counterfactual gain is 4.9 points (95% CI [2.0, 7.9]).

Question-time method	All (%)	C (%)	k tokens/q
Read identified section	73.7	38.6	6.4
Execute stored program (OTF-Tools)	74.9	43.5	8.1

hosted endpoints, which includes provider load. Memory use, program storage, and complete build wall time were not measured.

Code, benchmark manifests, prompts, and per-question predictions can be found here: Anonymous GitHub

5.2 What does a compiled program add?

Before comparing full systems, we separate two possible reasons OTF-Tools might succeed: finding the right records and computing an answer from them. Both variants in Table 2 use the same Navigation Map and the same question-specific section. One variant asks the LLM to answer after reading that section; the other runs an accepted simulator and uses the computed outcomes.

Reading versus computation. With Qwen3.6-27B on all 2,166 physics questions, reading the section achieves 73.7% overall accuracy. Running the simulator increases this to 74.9%. The gain is more pronounced on counterfactual questions: accuracy rises from 38.6% to 43.5%, a 4.9-point improvement (95% CI [2.0, 7.9]), for 1.7k additional amortized model tokens per question. The comparison shows that direct access to the records accounts for much of the overall performance, while executable replay adds value when the inputs must be changed. On the smaller E1-core subset, the counterfactual gain is 1.6 points and is not statistically significant; query-time agents are more accurate there but spend two to three times as many tokens (§5.8).

Other components. The supplement reports three additional physics ablations. Recomputing the entire history rather than keeping unaffected recorded history reduces Qwen accuracy by 6.3 points ([4.9, 7.7]), including 7.8 on counterfactual and 17.7 on explanatory questions. Disabling the cross-document program library reduces accuracy by 1.9 points. Replacing generated code with an oracle physics simulator adds only 0.2 points with Qwen but 6.1 points with gpt-oss-20b, where no generated physics program was accepted. These results isolate the value of preserving history, reusing successful code, and obtaining a sufficiently accurate simulator. They also identify a counterexample: on Qwen manufacturing, reading-only answers reach 74.7% overall and 64.0% counterfactual accuracy, compared with 69.0% and 58.7% when using the generated-program pipeline.

5.3 Accuracy versus inference cost

We next compare complete systems, counting model tokens spent on building indexes, generating programs, revising failed attempts, and answering questions.

Physics. Table 3 reports accuracy on E1. With gpt-oss-20b, OTF-Tools reaches 67.8%, compared with 61.2% for full-context prompting and 53.2% for PageIndex. With Qwen3.6-27B, it reaches 74.9%, compared with 67.8% and 76.7%, respectively; the 1.8-point difference from PageIndex is statistically supported (95% CI $[-3.2, -0.4]$). With Gemma-4-31B, OTF-Tools reaches 82.5%, effectively matching full context at 82.6% and exceeding PageIndex at 77.1%.

These results use substantially fewer model tokens. Table 4 shows that OTF-Tools consumes 7–9k amortized tokens per question, versus 15–34k for PageIndex and 59–72k for full context. At Qwen's hosted list price, 1,000 physics questions cost $10.2 with OTF-Tools, $25.1 with PageIndex, and $36.3 with full context. Token and dollar costs need not agree across models: for gpt-oss, the higher price of generated tokens makes agent and OTF-Tools dollar costs similar despite the latter's lower token use. On the matched 694-question E1-core subset, Qwen agents obtain higher

Table 3. Physics accuracy (%) by question type: descriptive (D), explanatory (E), predictive (P), counterfactual (C). OTF-Tools is the most accurate system with gpt-oss-20b, within two points of PageIndex with Qwen3.6-27B, and level with full context with Gemma-4-31B, while using far fewer tokens (Table 4). The upper block covers all 2,166 E1 questions. The agents ran on E1-core (five of the 15 documents, 694 questions), so the lower block repeats OTF-Tools on exactly those questions. §ReAct-C with Qwen was served from a local NVFP4 checkpoint rather than the hosted endpoint. *Partial Gemma runs, scored on the questions answered before the deadline: GraphRAG 428 and ReCode 314 of 694; OTF-Tools scores 76.9% and 77.1% on the same questions.

	gpt-oss-20b					Qwen3.6-27B					Gemma-4-31B				
System	D	E	P	C	All	D	E	P	C	All	D	E	P	C	All
E1, all 2,166 questions															
Full context	95.4	37.0	26.7	42.6	61.2	96.7	46.4	84.8	35.0	67.8	97.3	**89.9**	**98.2**	46.6	**82.6**
RAG	68.1	40.2	30.4	34.3	49.1	73.6	40.9	54.8	27.4	52.2	71.3	70.8	48.4	30.7	58.5
PageIndex	89.3	33.1	12.4	30.1	53.2	**98.2**	**69.8**	93.1	42.4	**76.7**	97.6	68.4	97.2	44.6	77.1
OTF-Tools	**96.6**	**48.5**	**57.6**	**43.7**	**67.8**	97.3	61.2	**96.3**	**43.5**	74.9	**99.0**	85.0	82.0	**54.2**	82.5
E1-core, 694 questions															
GraphRAG	24.4	15.9	5.9	8.1	16.1	72.3	41.8	26.5	20.5	46.5	60.8*	76.5*	6.7*	20.0*	48.4*
ReAct-R	80.1	31.2	11.8	27.6	47.4	97.4	82.4	**100.0**	**42.2**	79.3	98.9	85.3	98.5	40.0	79.8
ReAct-C	85.6	35.9	23.5	29.7	52.4	97.0§	**91.8**§	95.6§	**42.2**§	**81.0**§	98.9	79.4	**100.0**	44.9	79.8
ReCode	15.1	8.2	5.9	3.2	9.4	74.9	52.9	94.1	31.9	59.9	80.2*	34.2*	63.3*	14.6*	50.3*
OTF-Tools	**96.7**	**55.9**	**63.2**	**37.8**	**67.7**	**98.9**	62.4	97.1	36.8	73.2	**99.6**	**86.5**	77.9	**48.1**	**80.5**
Oracle model (E1)	97.6	54.4	88.0	48.6	73.9	97.2	63.0	94.5	43.5	75.1	–	–	–	–	–

Table 4. Cost of physics questions. Tokens per question include each system's setup spread over its questions. Calls, 95th-percentile latency, and dollars per 1,000 questions at the provider's list price are shown for Qwen3.6-27B; the supplement gives dollars for every LLM and workload. OTF-Tools uses the fewest tokens of any system except single-shot RAG, with every LLM, and makes two or three model calls per question. ‡E1-core. §Local checkpoint: latency not comparable, dollars at the hosted rate. *Partial run. PageIndex's internal calls are not counted.

	k tokens / question			Qwen3.6-27B		
System	gpt-oss	Qwen	Gemma	Calls	p95 s	$/1k
Full context	58.7	70.4	72.2	1.0	113	36.3
RAG	2.0	3.4	3.5	1.0	85	6.0
PageIndex	15.5	33.7	25.9	–	308	25.1
GraphRAG‡	33.6	47.2	–*	1.0	80	70.7
ReAct-R‡	11.1	16.6	16.2	5.0	258	19.1
ReAct-C‡	10.8	20.8§	17.1	5.1§	–§	23.9§
ReCode‡	11.5	35.0	–*	5.1	352	40.4
OTF-Tools	**9.3**	**8.1**	**7.2**	2.5	119	**10.2**

accuracy by making more calls and spending more tokens.

Across workloads. Table 5 extends the accuracy comparison to manufacturing, IT operations, and finance. OTF-Tools uses the fewest tokens beyond single-shot RAG in 14 of 17 reported settings. More importantly, across the 14 accuracy–token panels with comparable results, no evaluated alternative achieves both higher accuracy and lower model-token cost (supplement, Figure 2). This is a measured Pareto tradeoff, not a claim that OTF-Tools always has the highest accuracy. For example, with gpt-oss on manufacturing counterfactuals, OTF-Tools scores 74.7% against 46.7% for full context and 42.7% for PageIndex. With Qwen on IT counterfactuals, it scores 37.3% against 8.0% and 13.3%. In Qwen finance, ReAct-R is both cheaper in dollars and more accurate, illustrating why price-weighted comparisons are also necessary.

5.4 What happens when the document grows?

E2 asks the same 1,067 physics questions as E1 but places all their fixed-size, named video records inside one much longer document. It therefore tests whether adding unrelated records changes the cost or accuracy of answering a question about a known section.

Table 6 shows a clear difference. Full-context prompting can include only the first 106 pages of the 784-page E2 document under the common 131k context limit and loses 23–27 accuracy points. RAG loses 14–20 points. PageIndex loses 4–7 points and its token use grows to 168–196k per question. OTF-Tools instead changes by only −0.3 points with gpt-oss and −0.9 with Qwen, consuming 6–10k amortized tokens per question. With gpt-oss, it reaches 45.7% compared with PageIndex's 24.2%; with Qwen, the systems score 51.1% and 51.6%, respectively, but OTF-Tools uses 19× fewer tokens. Thus adding unrelated scopes has little effect when the question already identifies the required record group. Qwen's context-window size is scaled up to a 262,144 tokens for E2.

5.5 When does building a program pay off?

Compilation pays off when many questions or documents can share a successful program. Table 7 reports how many

Table 5. Question accuracy (%) on every workload: overall (All) and on counterfactual questions (C). Columns are separate workloads. E2 repeats E1's explanatory and counterfactual questions in a document fifteen times larger. OTF-Tools stays competitive overall while using the fewest tokens (Table 4), and it has the best counterfactual accuracy in 7 of the 14 columns shown. Bold marks the best value in each column for each LLM. ‡Agents ran physics on E1-core (694 questions), where OTF-Tools scores 67.7/37.8 (gpt-oss), 73.2/36.8 (Qwen) and 80.5/48.1 (Gemma). §Local checkpoint. –: not run. Unanswered questions count as wrong.

		Physics E1		Physics E2		Manufacturing		IT operations		Finance	
LLM	System	All	C	All	C	All	C	All	C	All	C
gpt-oss-20b	Full context	61.2	42.6	13.2	13.9	54.3	46.7	18.3	4.0	65.3	63.3
	RAG	49.1	34.3	17.2	11.4	25.3	38.7	16.7	5.3	64.0	65.3
	PageIndex	53.2	30.1	24.2	24.7	46.0	42.7	32.0	**8.0**	52.3	49.3
	ReAct-R	47.4‡	27.6‡	–	–	**68.7**	69.3	32.0	2.7	66.0	**68.0**
	ReAct-C	52.4‡	29.7‡	–	–	64.3	61.3	**34.3**	**8.0**	65.0	66.0
	OTF-Tools	**67.8**	**43.7**	**45.7**	**43.3**	64.0	**74.7**	28.3	4.0	**69.0**	67.3
Qwen3.6-27B	Full context	67.8	35.0	17.6	15.3	79.0	73.3	43.0	8.0	82.7	79.3
	RAG	52.2	27.4	20.1	14.8	27.3	40.0	26.0	20.0	76.3	77.3
	PageIndex	76.7	42.4	**51.6**	**41.2**	77.3	65.3	45.3	13.3	83.0	81.3
	ReAct-R	79.3‡	42.2‡	–	–	93.7	92.0	50.7	20.0	**84.7**	**84.0**
	ReAct-C	**81.0**‡§	42.2‡§	–	–	**97.0**	**97.3**	**57.3**	24.0	83.7	82.0
	OTF-Tools	74.9	**43.5**	51.1	40.1	69.0	58.7	52.3	**37.3**	83.7	82.0
Gemma-4-31B	Full context	**82.6**	46.6	–	–	83.3	**74.7**	44.0	13.3	78.0	74.0
	RAG	58.5	30.7	–	–	30.0	53.3	24.7	14.7	73.0	74.0
	PageIndex	77.1	44.6	–	–	80.3	61.3	50.7	12.0	79.7	80.0
	ReAct-R	79.8‡	40.0‡	–	–	82.3	61.3	51.0	16.0	80.0	80.0
	ReAct-C	79.8‡	44.9‡	–	–	**86.7**	72.0	**52.0**	14.7	**81.7**	**82.0**
	OTF-Tools	82.5	**54.2**	–	–	82.0	70.7	50.3	**17.3**	80.0	78.7

Table 6. The same 1,067 questions asked of 54k-token documents (E1) and of one 838k-token document (E2). Δ is the change in accuracy; tokens are per question on E2, setup included. OTF-Tools's accuracy and cost barely move, while full context and RAG lose 14–27 points and PageIndex's cost rises to 168–196k tokens per question. Full context keeps the first 106 of 784 pages on E2.

	gpt-oss-20b				Qwen3.6-27B			
	E1	E2	Δ	k tok	E1	E2	Δ	k tok
Full context	39.9	13.2	−26.7	113.0	40.5	17.6	−22.9	138.9
RAG	37.1	17.2	−19.9	2.1	33.9	20.1	−13.8	4.1
PageIndex	31.6	24.2	−7.4	168.1	**55.6**	**51.6**	−4.0	196.0
OTF-Tools	**46.0**	**45.7**	**−0.3**	6.1	52.0	51.1	**−0.9**	10.3

distinct programs were built and how many documents reused them. With Qwen3.6-27B, four generated programs serve all 25 process documents: one physics program alone serves fifteen documents. The physics setup consumes 0.17M tokens, and its cost is recovered against full-context prompting after three questions. Both its setup and query costs are lower than PageIndex's in that setting. With Gemma, three programs serve 16 documents and physics setup is recovered after 19 questions.

The outcome depends on generation success. gpt-oss-20b does not produce an accepted physics simulator; its 9.95M-token attempted physics setup raises break-even to 184 questions against full context and 834 against PageIndex. This cost is included even though answers use the reading fallback. A library ablation further demonstrates the value of reuse: without the library, Qwen must generate anew for each document, the share of questions answered by programs falls from 59% to 36%, and overall accuracy declines by 1.9 points ([0.4, 3.4]). Successful reuse therefore improves both cost and program availability, while unsuccessful synthesis is a measurable setup expense.

5.6 Question-time predictability

Once preparation is complete, OTF-Tools uses 2.0–3.0 LLM calls per question. The agents average 2–8 and can make up to twelve. On E1, OTF-Tools' 95th-percentile query latency is 19, 119, and 252 seconds across the three models, compared with 47, 308, and 1,020 seconds for PageIndex. At most 0.3% of OTF-Tools answers cannot be parsed into the required format, compared with maxima of 34% for full context and 17% for PageIndex. This fixed execution path produces more predictable call counts and answer formats;

Table 7. Program reuse on the 25 process documents (15 physics, 5 manufacturing, 5 IT operations), and the cost of building programs for physics. Generation success varies markedly by LLM; unsupported documents use a reading fallback. With Qwen3.6-27B, the physics setup is recovered against full context after three questions. Break-even uses Equation 4.

	Documents (of 25)			Physics setup	
LLM	Program written	Program reused	No program	Mtok	Break-even (questions)
gpt-oss-20b	3	4	18	9.95	184
Qwen3.6-27B	4	21	0	0.17	3
Gemma-4-31B	3	13	9	1.20	19

absolute latency comparisons also reflect hosted-provider load (§7).

5.7 When no usable simulator is available

The system can still answer directly from the identified section if compilation fails or the question does not require a time-indexed simulation. This explains an important result for the smallest model: gpt-oss-20b accepts no physics program across fifteen documents, and 97.6% of its non-descriptive physics questions use the reading fallback. Nevertheless, it reaches 67.8% physics accuracy, matching Qwen full-context prompting at approximately one eighth of the model-token cost. The advantage in this case comes from structured record access, not execution of an accepted simulator.

Finance provides a separate fallback control. OTF-Tools scores 69.0%, 83.7%, and 80.0% with the three models, respectively, despite the absence of a suitable process simulation in most cases. Gemma manufacturing likewise reaches 82.0% with no accepted program on its five documents. Together with Qwen's successful compilation across all 25 process documents, these results show how a single pipeline combines reliable scoped reading with additional computation when generation succeeds.

5.8 Where executable replay remains challenging

The results also reveal when query-time reasoning or reading is preferable. First, on Qwen's smaller E1-core physics subset, ReAct-C achieves 81.0% accuracy and ReAct-R 79.3%, versus 73.2% for OTF-Tools. The agents, which share the Navigation Map, are particularly effective on explanatory questions but spend two to four times as many tokens. They were not evaluated on E2.

Second, the generated-program pipeline is substantially less accurate on Qwen manufacturing: 69.0% overall versus 97.0% for ReAct-C and 74.7% for the reading-only variant. The deficit spans explanatory questions (48.0% versus ReAct-C's 93.3%), predictive questions (70.7% versus 97.3%), and counterfactual questions (58.7% versus 97.3%). Removing an order cannot directly express changes to dispatch policy, and the predictive and counterfactual gaps show that the limitations extend beyond policy explanations.

Third, the screening test can accept a simulator whose downstream predictions remain weak. Gemma's weakest accepted physics program has forecast F1 of 0.42; on the two documents it serves, non-descriptive accuracy is approximately 20 points below full context. These results define the principal boundary of the approach: reusable code saves inference when it captures the necessary process behavior, but a weak approximation can propagate errors across many questions.

6 Related Work

Reading and retrieving documents. RAG selects relevant text for each question (Lewis et al., 2020); RAPTOR, PageIndex, and GraphRAG organize documents into hierarchies, section trees, or entity graphs (Sarthi et al., 2024; Zhang & Tang, 2025; Edge et al., 2024). Long-context prompting supplies more of the original document, although models may use distant evidence unevenly (Liu et al., 2024). OTF-Tools also relies on structured document access, but it preserves records as inputs to a process simulator. This is a distinction in downstream use, not a claim that retrieval systems cannot preserve table structure.

Generating code for questions. Program of Thoughts and PAL use code to compute answers (Chen et al., 2023; Gao et al., 2023); ReAct, CodeAct, and ReCode generate tool actions or code as questions are processed (Yao et al., 2023; Wang et al., 2024b; Yu et al., 2025). OTF-Tools instead prepares a program in advance and runs it for multiple questions. The ReAct-C baseline shares its Navigation Map and interpreter, making the additional cost of query-time code generation measurable.

Reusable tools and process models. LATM, CRAFT, CREATOR, ToolMaker, and Voyager create code tools or executable skills that can be reused (Cai et al., 2024; Yuan et al., 2024; Qian et al., 2023; Wölflein et al., 2025; Wang et al., 2024a). WorldCoder and GIF-MCTS develop executable world models from interaction or trajectories (Tang et al., 2024; Dainese et al., 2024). OTF-Tools constructs its candidate model from an existing record, checks predicted against recorded outcomes, and tests compatibility before reusing code on another document. Process mining discovers or checks models from event logs (van der Aalst, 2016); here the immediate objective is answering questions. Think Before You Simulate and Executable Counterfactuals also use computational models for counterfactual reasoning (Ishay et al., 2024; Vashishtha et al., 2025).

7 LIMITATIONS

Our experiments focus on machine-readable process records for which each question identifies the relevant record group. The document-scale experiment tests the effect of adding such groups, rather than increasing the size of an individual group. Manufacturing and IT pair real input traces with outcomes generated under explicit policies, enabling controlled evaluation of changed inputs. Automatic scope identification, scanned sources, and incomplete histories fall outside this experimental setting. The program acceptance test measures agreement with recorded outcomes under masking and checks that predictions respond to input changes. Because the generating LLM can inspect the original document, this is a practical consistency screen rather than an independent guarantee of correct behavior under arbitrary interventions. Similarly, the current execution engine supports a defined set of input and parameter changes and uses a heuristic replay starting point; more expressive policy interventions and earlier cross-entity effects require additional modeling.

8 CONCLUSION

We introduced process-record QA as a setting in which answering may require computing alternative process outcomes rather than retrieving recorded facts. OTF-Tools addresses this setting by preserving structured evidence, building executable approximations of the process, checking them against recorded behavior, and reusing them across questions and compatible documents. Experiments distinguish the two principal benefits: scoped record access reduces what the LLM must read, while executable replay raises counterfactual accuracy by 4.9 points on Qwen physics. The complete system uses 4.2× fewer model tokens than PageIndex there, with a 1.8-point accuracy difference; one accepted physics program serves fifteen documents and recovers its setup cost against full-context prompting after three questions. Accuracy remains nearly unchanged when the same questions are repackaged into a document fifteen times longer. Together, these results establish reusable process execution as a practical means of amortizing LLM inference for repeated questions over process records, and show why the quality of synthesized programs and the reading fallback are integral to the design.

REFERENCES

Cai, T., Wang, X., Ma, T., Chen, X., and Zhou, D. Large language models as tool makers. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2305.17126.

Chen, W., Ma, X., Wang, X., and Cohen, W. W. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023. URL https://arxiv.org/abs/2211.12588.

Dainese, N., Merler, M., Alakuijala, M., and Marttinen, P. Generating code world models with large language models guided by Monte Carlo tree search. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.15383.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., and Larson, J. From local to global: A graph RAG approach to query-focused summarization. arXiv:2404.16130, 2024. URL https://arxiv.org/abs/2404.16130.

Efron, B. and Tibshirani, R. J. *An Introduction to the Bootstrap*. Chapman and Hall, 1993.

Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., and Neubig, G. PAL: Program-aided language models. In *International Conference on Machine Learning*, 2023. URL https://arxiv.org/abs/2211.10435.

Gemma Team. Gemma 4: gemma-4-31B-it model card. Model card, 2026. URL https://huggingface.co/google/gemma-4-31B-it.

Ishay, A., Yang, Z., Lee, J., Kang, I., and Lim, D. Think before you simulate: Symbolic reasoning to orchestrate neural computation for counterfactual question answering. In *IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024. URL https://openaccess.thecvf.com/content/WACV2024/html/Ishay_Think_Before_You_Simulate_Symbolic_Reasoning_To_Orchestrate_Neural_Computation_WACV_2024_paper.html.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2005.11401.

Li, M., Feng, F., Zhang, H., He, X., Zhu, F., and Chua, T.-S. Learning to imagine: Integrating counterfactual thinking in neural discrete reasoning. In *Annual Meeting of the Association for Computational Linguistics*, pp. 57–69, 2022. URL https://aclanthology.org/2022.acl-long.5/.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *Transactions of the Asso-*

ciation for Computational Linguistics, 12:157–173, 2024. URL https://arxiv.org/abs/2307.03172.

OpenAI. gpt-oss-120b & gpt-oss-20b model card. arXiv:2508.10925, 2025. URL https://arxiv.org/abs/2508.10925.

Qian, C., Han, C., Fung, Y. R., Qin, Y., Liu, Z., and Ji, H. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023. URL https://arxiv.org/abs/2305.14318.

Qwen Team. Qwen3.6-27B. Model card, 2026. URL https://huggingface.co/Qwen/Qwen3.6-27B.

Reiss, C., Wilkes, J., and Hellerstein, J. L. Google cluster-usage traces: Format + schema. Technical report, Google, 2011.

Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., and Manning, C. D. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2401.18059.

Tang, H., Key, D., and Ellis, K. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2402.12275.

van der Aalst, W. M. P. *Process Mining: Data Science in Action*. Springer, 2 edition, 2016. doi: 10.1007/978-3-662-49851-4. URL https://doi.org/10.1007/978-3-662-49851-4.

Vashishtha, A., Dai, Q., Mei, H., Sharma, A., Tan, C., and Peng, H. Executable counterfactuals: Improving LLMs' causal reasoning through code. arXiv:2510.01539, 2025. URL https://arxiv.org/abs/2510.01539.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a. URL https://arxiv.org/abs/2305.16291.

Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning*, 2024b. URL https://arxiv.org/abs/2402.01030.

Wasi, A. T., Islam, M. S., and Akib, A. R. SupplyGraph: A benchmark dataset for supply chain planning using graph neural networks. arXiv:2401.15299, 2024. URL https://arxiv.org/abs/2401.15299.

Wölflein, G., Ferber, D., Truhn, D., Arandjelović, O., and Kather, J. N. LLM agents making agent tools. In *Annual Meeting of the Association for Computational Linguistics*, 2025. URL https://arxiv.org/abs/2502.11705.

Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., and Nie, J.-Y. C-Pack: Packed resources for general chinese embeddings. arXiv:2309.07597, revised version, 2024. URL https://arxiv.org/abs/2309.07597.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

Yi, K., Gan, C., Li, Y., Kohli, P., Wu, J., Torralba, A., and Tenenbaum, J. B. CLEVRER: Collision events for video representation and reasoning. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/abs/1910.01442.

Yu, Z., Zhang, J., Su, H., Zhao, Y., Wu, Y., Deng, M., Xiang, J., Lin, Y., Tang, L., Luo, Y., Liu, B., and Wu, C. ReCode: Unify plan and action for universal granularity control. arXiv:2510.23564, 2025. URL https://arxiv.org/abs/2510.23564.

Yuan, L., Chen, Y., Wang, X., Fung, Y. R., Peng, H., and Ji, H. CRAFT: Customizing LLMs by creating and retrieving from specialized toolsets. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2309.17428.

Zhang, M. and Tang, Y. PageIndex: Next-generation vectorless, reasoning-based RAG. PageIndex technical introduction, September 2025. URL https://pageindex.ai/blog/pageindex-intro.

Zhu, F., Lei, W., Huang, Y., Wang, C., Zhang, S., Lv, J., Feng, F., and Chua, T.-S. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Annual Meeting of the Association for Computational Linguistics*, pp. 3277–3287, 2021. URL https://aclanthology.org/2021.acl-long.254/.

AI USAGE DISCLOSURE

Generative AI tools were used to improve manuscript clarity, wording, presentation, and review experimental results for potential inconsistencies. All AI-assisted revisions were reviewed by the authors, who retain full responsibility for the manuscript's content, results, and conclusions.